



You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Bar Widget

Bar widgets are components that appear in the Noctalia bar (top, bottom, left, or right). They provide quick access to information and actions.

Basic Structure

A bar widget is a QML file that must follow this structure. The pattern uses an Item root with a centered visualCapsule Rectangle to ensure click areas extend to the full bar height while keeping the visual content properly sized:

```
import QtQuick
import QtQuick.Layouts
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Commons
import qs.Widgets
```

```
Item {
    id: root
```

```
// Plugin API (injected by PluginService)
property var pluginApi: null
```

```
// Required properties for bar widgets
property ShellScreen screen
property string widgetId: ""
property string section: ""
property int sectionWidgetIndex: -1
property int sectionWidgetsCount: 0
```

```
// Per-screen bar properties (for multi-monitor and vertical bar support)
readonly property string screenName: screen?.name ?? ""
readonly property string barPosition: Settings.getBarPositionForScreen(screenName)
readonly property bool isBarVertical: barPosition === "left" || barPosition === "right"
readonly property real capsuleHeight: Style.getCapsuleHeightForScreen(screenName)
readonly property real barFontSize: Style.getBarFontSizeForScreen(screenName)
```

```
// Content dimensions (visual capsule size)
readonly property real contentWidth: content.implicitWidth + Style.marginM * 2
readonly property real contentHeight: capsuleHeight
```

```
// Widget dimensions (extends to full bar height for better click area)
implicitWidth: contentWidth
implicitHeight: contentHeight
```

```
// Visual capsule - centered within the full click area
Rectangle {
    id: visualCapsule
    x: Style.pixelAlignCenter(parent.width, width)
    y: Style.pixelAlignCenter(parent.height, height)
```

width: `root.contentWidth`

height: `root.contentHeight`

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
border.color: Style.capsuleBorderColor
```

```
border.width: Style.capsuleBorderWidth
```

```
// Your widget content here (centered in visualCapsule)
```

```
RowLayout {
```

```
    id: content
```

```
    anchors.centerIn: parent
```

```
    spacing: Style.marginS
```

```
    // ... content items
```

```
}
```

```
}
```

```
// MouseArea at root level for extended click area
```

```
MouseArea {
```

```
    id: mouseArea
```

```
    anchors.fill: parent
```

```
    hoverEnabled: true
```

```
    cursorShape: Qt.PointingHandCursor
```

```
    onClicked: {
```

```
        // Handle click
```

```
    }
```

```
}
```

```
}
```

Extended Click Areas

The Item root with centered visualCapsule pattern ensures users can click slightly above or below the visible widget and still trigger the click event. This significantly improves usability, especially on high-density bars.

Required Properties

pluginApi

pluginApi is a property of the `BarWidget` class in the `BarWidget` plugin.

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
property var pluginApi: null
```

screen

The `ShellScreen` this widget is displayed on (for multi-monitor support).

```
property ShellScreen screen
```

widgetId

Unique identifier for this widget instance.

```
property string widgetId: ""
```

section

The bar section this widget is in: "left", "center", or "right".

```
property string section: ""
```

sectionWidgetIndex

The zero-based index of this widget within its bar section. Used to access per-instance settings and to build unique identifiers for service registration.

```
property int sectionWidgetIndex: -1
```

sectionWidgetsCount

The total number of widgets in this widget's bar section.

```
property int sectionWidgetsCount: 0
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Sizing

Bar widgets use `contentWidth` and `contentHeight` properties to define the visual capsule size, while `implicitWidth` and `implicitHeight` control the click area. Use per-screen properties to support multi-monitor setups and per-screen bar settings:

```
// Per-screen bar properties (required for proper multi-monitor support)
readonly property string screenName: screen?.name ?? ""
readonly property string barPosition: Settings.getBarPositionForScreen(screenName)
readonly property bool isBarVertical: barPosition === "left" || barPosition === "right"
readonly property real capsuleHeight: Style.getCapsuleHeightForScreen(screenName)
readonly property real barFontSize: Style.getBarFontSizeForScreen(screenName)

// Define visual content size using per-screen capsuleHeight
readonly property real contentWidth: content.implicitWidth + Style.marginM * 2
readonly property real contentHeight: capsuleHeight

// Click area matches content (bar loader may extend it)
implicitWidth: contentWidth
implicitHeight: contentHeight
```

For vertical bar support (left/right positions):

```
// Per-screen bar properties
```

```
readonly property string screenName: screen?.name ?? ""
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
readonly property bool isBarVertical: barPosition === "left" || barPosition === "right"
```

```
readonly property real capsuleHeight: Style.getCapsuleHeightForScreen(screenName)
```

```
readonly property real barFontSize: Style.getBarFontSizeForScreen(screenName)
```

```
// Swap width/height for vertical bars
```

```
readonly property real contentWidth: isBarVertical ? capsuleHeight : content.implicitWidth +  
Style.marginM * 2
```

```
readonly property real contentHeight: isBarVertical ? content.implicitHeight + Style.marginM * 2 :  
capsuleHeight
```

```
implicitWidth: contentWidth
```

```
implicitHeight: contentHeight
```

Per-Screen Properties

Always use `Style.getCapsuleHeightForScreen(screenName)` and `Style.getBarFontSizeForScreen(screenName)` instead of `Style.capsuleHeight` or `Style.barFontSize`. This ensures your widget respects per-monitor bar settings and scales correctly on each screen.

Styling

Use Noctalia's built-in styling system for consistency:

Required Styling

Bar widgets **must** have a background and an outline for proper visual integration with the bar. Apply these to the `visualCapsule Rectangle`:

```
Rectangle {  
  id: visualCapsule
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
// Required: Background color with hover support  
color: mouseArea.containsMouse ? Color.mHover : Style.capsuleColor  
  
// Required: Outline border  
border.color: Style.capsuleBorderColor  
border.width: Style.capsuleBorderWidth  
  
// Required: Rounded corners  
radius: Style.radiusL  
}
```

Without these properties, the widget may not display correctly or may not integrate properly with the bar's visual style.

Colors

```
import qs.Commons  
  
Rectangle {  
  id: visualCapsule  
  
  // Surface colors (for visualCapsule background)  
  color: Color.mSurface  
  color: Color.mSurfaceVariant  
  color: Style.capsuleColor // Recommended for bar widgets  
  
  // Text colors  
  NText {  
    color: Color.mOnSurface  
    color: Color.mOnSurfaceVariant  
    color: Color.mPrimary  
  }  
}
```

Spacing and Sizes

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
ColumnLayout {  
  spacing: Style.marginS // Small spacing  
  spacing: Style.marginM // Medium spacing  
  spacing: Style.marginL // Large spacing  
  
  NIcon {  
    pointSize: Style.fontSizeS // Small icon  
    pointSize: Style.fontSizeM // Medium icon  
    pointSize: Style.fontSizeL // Large icon  
  }  
}  
  
Rectangle {  
  radius: Style.radiusS // Small radius  
  radius: Style.radiusM // Medium radius (recommended)  
  radius: Style.radiusL // Large radius  
}
```

Typography


```
// Define per-screen font size at root level  
readonly property string screenName: screen?.name ?? ""
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
NText {  
  // For bar widgets, use per-screen barFontSize for consistent sizing  
  pointSize: barFontSize // Recommended for bar widget text  
  
  // Other available sizes (for panels, dialogs, etc.)  
  pointSize: Style.fontSizeXS  
  pointSize: Style.fontSizeS  
  pointSize: Style.fontSizeM  
  pointSize: Style.fontSizeL  
  
  font.weight: Font.Light  
  font.weight: Font.Normal  
  font.weight: Font.Medium  
  font.weight: Font.Bold  
}
```

Tip

Always use `Style.getBarFontSizeForScreen(screenName)` for text in bar widgets to ensure consistent sizing with other bar elements and proper scaling per-screen.

Using Settings

Access plugin settings via `pluginApi`:

```
// Get setting with fallback to default  
readonly property string message:
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
pluginApi?.manifest?.metadata?.defaultSettings?.message ||  
"Default Message"
```

```
// Update and save setting  
function updateMessage(newMessage) {  
    pluginApi.pluginSettings.message = newMessage  
    pluginApi.saveSettings()  
}
```

Interactions

Click Events

```
MouseArea {  
    anchors.fill: parent  
    cursorShape: Qt.PointingHandCursor  
  
    onClicked: {  
        // Open plugin panel near this widget  
        pluginApi.openPanel(root.screen, root)  
  
        // Or perform action  
        performAction()  
    }  
}
```

Hover Effects

Use a property binding on the visualCapsule color instead of onEntered/onExited handlers for cleaner, more reliable hover effects:

```
Rectangle {  
    id: visualCapsule
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
// Hover effect via binding - cleaner than onEntered/onExited  
color: mouseArea.containsMouse ? Color.mHover : Style.capsuleColor  
}  
  
MouseArea {  
    id: mouseArea  
    anchors.fill: parent  
    hoverEnabled: true  
    cursorShape: Qt.PointingHandCursor  
  
    onClicked: {  
        // Handle click  
    }  
}
```

Tip

Using a binding for color changes is more robust than imperative `onEntered/onExited` handlers because:

- The color always reflects the current hover state
- No risk of stuck hover states if the mouse exits unexpectedly
- Works correctly when the widget is destroyed while hovered

Tooltips

```
import qs.Services.UI
import qs.Services.System
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
MouseArea {
    anchors.fill: parent
    hoverEnabled: true

    onEntered: {
        TooltipService.show(root, "Widget tooltip text", BarService.getTooltipDirection())
    }

    onExited: {
        TooltipService.hide()
    }
}
```

You can also use a function to build dynamic tooltip content:

```
function buildTooltip() {
    return "Status: " + (enabled ? "Active" : "Inactive")
}

MouseArea {
    anchors.fill: parent
    hoverEnabled: true

    onEntered: {
        TooltipService.show(root, buildTooltip(), BarService.getTooltipDirection())
    }

    onExited: {
        TooltipService.hide()
    }
}
```

The `BarService.getTooltipDirection()` function automatically determines the correct tooltip direction based on the bar's position (top, bottom, left, or right).

Context Menus

API Reference | Introduction | Getting Started | Examples | Plugins | Development | Changelog

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Services.UI
import qs.Widgets
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
Item {
    id: root

    property var pluginApi: null
    property ShellScreen screen
    // ... other properties

    // Per-screen bar properties
    readonly property string screenName: screen?.name ?? ""
    readonly property real capsuleHeight: Style.getCapsuleHeightForScreen(screenName)

    readonly property real contentWidth: content.implicitWidth + Style.marginM * 2
    readonly property real contentHeight: capsuleHeight

    implicitWidth: contentWidth
    implicitHeight: contentHeight

    Rectangle {
        id: visualCapsule
        x: Style.pixelAlignCenter(parent.width, width)
        y: Style.pixelAlignCenter(parent.height, height)
        width: root.contentWidth
        height: root.contentHeight
        color: mouseArea.containsMouse ? Color.mHover : Style.capsuleColor
        radius: Style.radiusL
        border.color: Style.capsuleBorderColor
        border.width: Style.capsuleBorderWidth

        RowLayout {
            id: content
            anchors.centerIn: parent
            // ... content
        }
    }

    NPopupContextMenu {
```

id: contextMenu

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
"label": pluginApi?.tr("menu.refresh") || "Refresh",
"action": "refresh",
"icon": "refresh"
},
{
"label": pluginApi?.tr("menu.settings") || "Settings",
"action": "settings",
"icon": "settings"
}
]
```

```
onTriggered: action ⇒ {
    // Always close the menu first
    contextMenu.close();
    PanelService.closeContextMenu(screen);

    // Handle actions
    if (action === "refresh") {
        // Perform refresh
    } else if (action === "settings") {
        BarService.openPluginSettings(screen, pluginApi.manifest);
    }
}
}
```

```
MouseArea {
    id: mouseArea
    anchors.fill: parent
    hoverEnabled: true
    cursorShape: Qt.PointingHandCursor
    acceptedButtons: Qt.LeftButton | Qt.RightButton

    onClicked: (mouse) ⇒ {
        if (mouse.button === Qt.LeftButton) {
            pluginApi?.openPanel(root.screen, root)
        } else if (mouse.button === Qt.RightButton) {
```

```
PanelService.showContextMenu(contextMenu, root, screen);
}
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
}
```

Context Menu Model

Each item in the model array is an object with these properties:

Property	Type	Description
label	string	Display text for the menu item
action	string	Action identifier passed to onTriggered
icon	string	Tabler icon name (optional)
enabled	bool	Whether the item is clickable (default: true)

```
model: [
  { "label": "Active Item", "action": "active", "icon": "check" },
  { "label": "Disabled Item", "action": "disabled", "icon": "x", "enabled": false }
]
```

Using NIconButton

If your widget extends NIconButton, use the built-in onRightClicked signal:


```
import qs.Services.UI
```

```
import qs.Widgets
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
NIconButton {  
  id: root  
  
  property var pluginApi: null  
  property ShellScreen screen  
  
  icon: "my-icon"  
  
  onClicked: {  
    pluginApi?.openPanel(root.screen, root)  
  }  
  
  onRightClicked: {  
    PanelService.showContextMenu(contextMenu, root, screen);  
  }  
  
  NPopupMenu {  
    id: contextMenu  
  
    model: [  
      {  
        "label": I18n.tr("actions.widget-settings"),  
        "action": "widget-settings",  
        "icon": "settings"  
      }  
    ]  
  
    onTriggered: action ⇒ {  
      contextMenu.close();  
      PanelService.closeContextMenu(screen);  
  
      if (action === "widget-settings") {  
        BarService.openPluginSettings(screen, pluginApi.manifest);  
      }  
    }  
  }  
}
```

```
}
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

Always use `PanelService.showContextMenu()` to open context menus and call both `contextMenu.close()` and `PanelService.closeContextMenu(screen)` in the `onTriggered` handler. This ensures proper behavior across all Wayland compositors (Hyprrland, Sway, Niri, labwc, etc.).

Opening Panels

If your plugin provides a panel, open it from the widget:

```
MouseArea {  
    anchors.fill: parent  
    onClicked: {  
        if (pluginApi) {  
            pluginApi.openPanel(root.screen, root)  
        }  
    }  
}
```

Tip

Passing `root` (or this from `onClicked`) as the second parameter to `openPanel()` makes the panel open near the widget, similar to how core Noctalia panels work.

Using Noctalia Widgets

Noctalia provides many pre-built widgets:

Icons

```
import qs.Widgets
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
icon: "heart" // Tabler icon name
color: Color.mPrimary
applyUiScale: true // Automatically scale with UI
}
```

Text

```
NText {
  text: "Hello World"
  color: Color.mOnSurface
  pointSize: barFontSize // Use per-screen barFontSize for bar widgets
  font.weight: Font.Medium
}
```

Buttons

```
NIconButton {
  icon: "settings"
  onClicked: {
    // Handle click
  }
}
```

Accessing Services

Plugins can use Noctalia services:

Toast Notifications

```
import qs.Services.UI
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
ToastService.showError("Error message")
```

Logging

```
import qs.Commons
```

```
Component.onCompleted: {  
  Logger.i("MyPlugin", "Widget loaded")  
  Logger.d("MyPlugin", "Debug info:", someValue)  
  Logger.w("MyPlugin", "Warning message")  
  Logger.e("MyPlugin", "Error occurred")  
}
```

Settings Access

```
import qs.Commons
```

```
// Access global Noctalia settings  
readonly property string barPosition: Settings.data.bar.position  
readonly property bool isDarkMode: Settings.data.ui.darkMode
```

Complete Examples

Simple Status Widget

```
import QtQuick
import QtQuick.Layouts
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Commons
import qs.Widgets
```

```
Item {
    id: root
```

```
    property var pluginApi: null
    property ShellScreen screen
    property string widgetId: ""
    property string section: ""
    property int sectionWidgetIndex: -1
    property int sectionWidgetsCount: 0
```

```
    // Per-screen bar properties
```

```
    readonly property string screenName: screen?.name ?? ""
    readonly property string barPosition: Settings.getBarPositionForScreen(screenName)
    readonly property bool isBarVertical: barPosition === "left" || barPosition === "right"
    readonly property real capsuleHeight: Style.getCapsuleHeightForScreen(screenName)
    readonly property real barFontSize: Style.getBarFontSizeForScreen(screenName)
```

```
    readonly property real contentWidth: row.implicitWidth + Style.marginM * 2
    readonly property real contentHeight: capsuleHeight
```

```
    implicitWidth: contentWidth
    implicitHeight: contentHeight
```

```
    Rectangle {
        id: visualCapsule
        x: Style.pixelAlignCenter(parent.width, width)
        y: Style.pixelAlignCenter(parent.height, height)
        width: root.contentWidth
        height: root.contentHeight
        color: Style.capsuleColor
        radius: Style.radiusL
        border.color: Style.capsuleBorderColor
        border.width: Style.capsuleBorderWidth
```

```
RowLayout {
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
  spacing: Style.marginS
```

```
  NIcon {
```

```
    icon: "check"
```

```
    color: Color.mPrimary
```

```
  }
```

```
  NText {
```

```
    text: "Ready"
```

```
    color: Color.mOnSurface
```

```
    pointSize: barFontSize
```

```
  }
```

```
}
```

```
}
```

```
}
```

Interactive Counter Widget

```
import QtQuick
import QtQuick.Layouts
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Commons
import qs.Widgets
import qs.Services.UI
```

```
Item {
    id: root
```

```
    property var pluginApi: null
    property ShellScreen screen
    property string widgetId: ""
    property string section: ""
    property int sectionWidgetIndex: -1
    property int sectionWidgetsCount: 0
```

```
    // Per-screen bar properties
```

```
    readonly property string screenName: screen?.name ?? ""
    readonly property string barPosition: Settings.getBarPositionForScreen(screenName)
    readonly property bool isBarVertical: barPosition === "left" || barPosition === "right"
    readonly property real capsuleHeight: Style.getCapsuleHeightForScreen(screenName)
    readonly property real barFontSize: Style.getBarFontSizeForScreen(screenName)
```

```
    property int count: pluginApi?.pluginSettings?.count || 0
```

```
    readonly property real contentWidth: row.implicitWidth + Style.marginM * 2
    readonly property real contentHeight: capsuleHeight
```

```
    implicitWidth: contentWidth
    implicitHeight: contentHeight
```

```
    Rectangle {
        id: visualCapsule
        x: Style.pixelAlignCenter(parent.width, width)
        y: Style.pixelAlignCenter(parent.height, height)
        width: root.contentWidth
        height: root.contentHeight
        color: mouseArea.containsMouse ? Color.mHover : Style.capsuleColor
```

radius: `Style.radiusL`

border.color: `Style.capsuleBorderColor`

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
RowLayout {
    id: row
    anchors.centerIn: parent
    spacing: Style.marginS

    NIcon {
        icon: "numbers"
        color: Color.mPrimary
    }

    NText {
        text: root.count.toString()
        color: Color.mOnSurface
        pointSize: barFontSize
        font.weight: Font.Bold
    }
}

MouseArea {
    id: mouseArea
    anchors.fill: parent
    hoverEnabled: true
    cursorShape: Qt.PointingHandCursor

    onClicked: {
        root.count++
        pluginApi.pluginSettings.count = root.count
        pluginApi.saveSettings()
        ToastService.showNotice("Count: " + root.count)
    }
}
```


Widget with Panel

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import QtQuick
import QtQuick.Layouts
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Commons
import qs.Widgets
```

```
Item {
    id: root
```

```
    property var pluginApi: null
    property ShellScreen screen
    property string widgetId: ""
    property string section: ""
    property int sectionWidgetIndex: -1
    property int sectionWidgetsCount: 0
```

```
    // Per-screen bar properties
```

```
    readonly property string screenName: screen?.name ?? ""
    readonly property string barPosition: Settings.getBarPositionForScreen(screenName)
    readonly property bool isBarVertical: barPosition === "left" || barPosition === "right"
    readonly property real capsuleHeight: Style.getCapsuleHeightForScreen(screenName)
    readonly property real barFontSize: Style.getBarFontSizeForScreen(screenName)
```

```
    readonly property string message:
        pluginApi?.pluginSettings?.message ||
        pluginApi?.manifest?.metadata?.defaultSettings?.message || ""
```

```
    readonly property real contentWidth: row.implicitWidth + Style.marginM * 2
    readonly property real contentHeight: capsuleHeight
```

```
    implicitWidth: contentWidth
    implicitHeight: contentHeight
```

```
    Rectangle {
        id: visualCapsule
        x: Style.pixelAlignCenter(parent.width, width)
        y: Style.pixelAlignCenter(parent.height, height)
        width: root.contentWidth
        height: root.contentHeight
```

color: `mouseArea.containsMouse ? Color.mHover : Style.capsuleColor`

radius: `Style.radiusL`

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
RowLayout {
    id: row
    anchors.centerIn: parent
    spacing: Style.marginS

    NIcon {
        icon: "noctalia"
    }

    NText {
        text: root.message
        color: Color.mOnSurface
        pointSize: barFontSize
    }
}

MouseArea {
    id: mouseArea
    anchors.fill: parent
    hoverEnabled: true
    cursorShape: Qt.PointingHandCursor

    onClicked: {
        if (pluginApi) {
            pluginApi.openPanel(root.screen, root)
        }
    }
}
```

Vertical Bar Support

Support vertical bars (left/right positions):

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import QtQuick
import QtQuick.Layouts
```

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
import qs.Commons
import qs.Widgets

Item {
    id: root

    property var pluginApi: null
    property ShellScreen screen
    property string widgetId: ""
    property string section: ""
    property int sectionWidgetIndex: -1
    property int sectionWidgetsCount: 0

    // Per-screen bar properties
    readonly property string screenName: screen?.name ?? ""
    readonly property string barPosition: Settings.getBarPositionForScreen(screenName)
    readonly property bool isBarVertical: barPosition === "left" || barPosition === "right"
    readonly property real capsuleHeight: Style.getCapsuleHeightForScreen(screenName)
    readonly property real barFontSize: Style.getBarFontSizeForScreen(screenName)

    readonly property real contentWidth: isBarVertical ? capsuleHeight : layout.implicitWidth +
    Style.marginM * 2
    readonly property real contentHeight: isBarVertical ? layout.implicitHeight + Style.marginM * 2 :
    capsuleHeight

    implicitWidth: contentWidth
    implicitHeight: contentHeight

    Rectangle {
        id: visualCapsule
        x: Style.pixelAlignCenter(parent.width, width)
        y: Style.pixelAlignCenter(parent.height, height)
        width: root.contentWidth
        height: root.contentHeight
        color: mouseArea.containsMouse ? Color.mHover : Style.capsuleColor
        radius: Style.radiusL
```

border.color: [Style.capsuleBorderColor](#)

border.width: [Style.capsuleBorderWidth](#)

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

id: layout

anchors.centerIn: parent

implicitWidth: [rowLayout.visible ? rowLayout.implicitWidth : colLayout.implicitWidth](#)

implicitHeight: [rowLayout.visible ? rowLayout.implicitHeight : colLayout.implicitHeight](#)

RowLayout {

id: rowLayout

visible: [!root.isBarVertical](#)

spacing: [Style.marginS](#)

NIcon {

icon: ["heart"](#)

color: [Color.mPrimary](#)

}

NText {

text: ["Widget"](#)

color: [Color.mOnSurface](#)

pointSize: barFontSize

}

}

ColumnLayout {

id: colLayout

visible: [root.isBarVertical](#)

spacing: [Style.marginS](#)

NIcon {

icon: ["heart"](#)

color: [Color.mPrimary](#)

}

NText {

text: ["Widget"](#)

color: [Color.mOnSurface](#)

pointSize: barFontSize

}

}

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

```
MouseArea {  
    id: mouseArea  
    anchors.fill: parent  
    hoverEnabled: true  
    cursorShape: Qt.PointingHandCursor  
  
    onClicked: {  
        // Handle click  
    }  
}
```

Best Practices

1. **Use the visual capsule pattern:** Item root with centered Rectangle visualCapsule for extended click areas
2. **Keep it small:** Bar widgets should be compact and unobtrusive
3. **Use consistent styling:** Stick to Noctalia's design system
4. **Use per-screen properties:** Always use Style.getCapsuleHeightForScreen(screenName) and Style.getBarFontSizeForScreen(screenName) instead of global values
5. **Handle missing data:** Always provide fallbacks for settings and data
6. **Respect dark mode:** Use Color.m* colors that adapt to theme
7. **Use binding for hover effects:** color: mouseArea.containsMouse ? Color.mHover : Style.capsuleColor
8. **Log important events:** Use Logger for debugging
9. **Test on vertical bars:** Ensure your widget works in all bar positions
10. **Optimize performance:** Avoid expensive operations in the widget

See Also

- [Getting Started](#) - Create your first plugin
- [Desktop Widget Development](#) - Create desktop widgets

You are viewing the **v4 (stable)** documentation. [Switch to v5 \(alpha\) →](#)

- [Plugin API](#) - Full API reference
- [Manifest Reference](#) - Plugin configuration